

Panduan Optimasi Performa SSIS

Best Practices untuk ETL Skala Besar

Daftar Isi

1. Query Sumber & Desain SQL	2
Hindari SELECT *	2
Gunakan Kueri <i>SARGable</i>	2
Hindari Cartesian / Accidental Cross Join	2
Optimasi View Bertumpuk / Unindexed View	2
Terapkan Filter Pushdown	3
Jaga Kebugaran Statistik & Indeks	3
Hindari Kalkulasi/Parsing <i>On-the-Fly</i> Per Baris	3
Tahapkan Pemrosesan PIVOT/Window Function	3
Gunakan <i>Partitioned Tables</i> Alih-alih UNION ALL	3
Gunakan Table Hint NOLOCK / <i>Read Uncommitted</i>	3
2. Konektivitas & Provider Data	5
Gunakan Provider Native OLE DB	5
Aktifkan DelayValidation = True	5
3. Desain & Komponen Data Flow (Canvas SSIS)	6
Eliminasi Komponen <i>Blocking</i>	6
Gunakan <i>Full Cache</i> pada Lookup	6
Manfaatkan <i>Shared Cache Connection Manager</i>	6
Pastikan Data Benar-Benar Terurut pada Merge Join	6
Pangkas Kolom Tidak Terpakai	6
Hindari Implicit Data Type Conversion	7
Hindari OLE DB Command Transformation	7
Optimasi Script Component (.NET)	7
Jalankan Data Flow Independen Secara Paralel	7
4. Konfigurasi Buffer, Memori & Eksekusi Pipeline	8
Sesuaikan Ukuran Buffer	8
Cegah Buffer Spooling ke Disk	8
Arahkan Folder Temporary ke Disk Terpisah	8
Optimasi Thread & Eksekusi Paralel	8
5. Tujuan / Loading ke Destination	9
Gunakan OLE DB Destination Fast Load	9
Sertakan Opsi TABLOCK	9
Perbesar Commit Size	9
Nonaktifkan Indeks & Constraints Sementara	9
Matikan Trigger Destination	9
6. Arsitektur, Orkestrasi Job & Lingkungan Server	10
Gunakan Pola Arsitektur Parent-Child	10

Terapkan <i>Checkpoints</i>	10
Atur Urutan Eksekusi Job (Job Chaining)	10
Gunakan Incremental Load	10
Optimasi tempdb Database	10
Alokasikan RAM/CPU Memadai	11
Pengecualian Antivirus (Exclusions)	11
Gunakan Logging Level Ringan di Production	11

1. Query Sumber & Desain SQL

Hindari SELECT *

Penjelasan: Mengambil semua kolom (termasuk kolom teks panjang) memperbesar ukuran baris data (*row width*), yang membuat buffer SSIS cepat penuh dan memicu I/O disk berlebih.

Contoh: `SELECT * FROM dbo.FactSales`

Solusi Terbaik: Pilih hanya kolom yang benar-benar akan diproses.

`SELECT CustomerID, OrderDate, TotalAmount FROM dbo.FactSales`

Gunakan Kueri *SARGable*

Penjelasan: Penggunaan fungsi atau manipulasi pada kolom di klausa WHERE/JOIN membuat SQL Server tidak bisa menggunakan indeks (*Index Scan* alih-alih *Index Seek*).

Contoh: `WHERE YEAR(OrderDate) = 2026` atau `WHERE UPPER(Status) = 'ACTIVE'`

Solusi Terbaik: Bandingkan nilai kolom secara langsung tanpa membalutnya dengan fungsi.

`WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01'`

Hindari Cartesian / Accidental Cross Join

Penjelasan: Menggabungkan tabel tanpa kondisi penggabungan yang spesifik memicu perkalian jumlah baris secara masif ($M \times N$).

Contoh: `SELECT * FROM Orders, Customers` (lupa menambahkan klausa pembandingan).

Solusi Terbaik: Gunakan sintaks INNER JOIN atau LEFT JOIN eksplisit lengkap dengan klausanya.

`SELECT o.OrderID, c.CustomerName
FROM dbo.Orders o
INNER JOIN dbo.Customers c ON o.CustomerID = c.CustomerID`

Optimasi View Bertumpuk / Unindexed View

Penjelasan: Mengambil data dari *view* biasa yang didalamnya memanggil *view* lain secara bertumpuk (*nested views*) memaksa mesin SQL mengeksekusi ulang seluruh logika dasar yang sangat kompleks.

Contoh: Kueri memanggil `vw_SalesSummary` yang dibuat dari `vw_OrderDetails`, di mana `vw_OrderDetails` menggabungkan 5 tabel operasional.

Solusi Terbaik: Buat *Indexed View* jika memungkinkan, atau ganti dengan kueri SQL langsung yang sudah dioptimasi.

Terapkan Filter Pushdown

Penjelasan: Mengalirkan seluruh data dari database ke SSIS Data Flow lalu menyaringnya menggunakan komponen Conditional Split membuang-buang kapasitas memori dan jaringan.

Contoh: Mengalirkan 50 juta baris data ke SSIS Canvas hanya untuk membuang data yang bukan tanggal hari ini.

Solusi Terbaik: Tapis data langsung di level database (*SQL Source Query*) menggunakan variabel SSIS.

```
SELECT * FROM Sales WHERE ModifiedDate >= @[User::LastRunDate]
```

Jaga Kebugaran Statistik & Indeks

Penjelasan: *Query Optimizer* memilih strategi eksekusi (*Execution Plan*) yang buruk jika estimasi jumlah baris tidak akurat akibat statistik data yang sudah kadaluarsa.

Contoh: Kueri menggunakan *Table Scan* pada tabel puluhan juta baris karena statistik membaca tabel tersebut masih kosong.

Solusi Terbaik: Buat *Maintenance Plan* rutin untuk UPDATE STATISTICS dan penataan ulang indeks (ALTER INDEX REBUILD/REORGANIZE).

Hindari Kalkulasi/Parsing On-the-Fly Per Baris

Penjelasan: Menjalankan operasi pemrosesan string atau rumus matematika rumit pada setiap baris di kueri sumber membebankan CPU server database secara intensif.

Contoh: SELECT SUBSTRING(Code, 1, 3) + CAST(ID AS VARCHAR) FROM BigTable

Solusi Terbaik: Simpan hasil kalkulasi dalam *computed column* berindeks saat data pertama kali masuk, atau lakukan kalkulasi berbasis set data.

Tahapkan Pemrosesan PIVOT/Window Function

Penjelasan: Operasi PIVOT/UNPIVOT atau *Window Function* ber-frame lebar (ROW_NUMBER() OVER (...)) pada volume data besar di dalam CTE memakan ruang tempdb yang sangat tinggi dan memicu proses pengurutan (*Sort*) yang mahal.

Contoh: ROW_NUMBER() OVER (PARTITION BY Region ORDER BY TransactionDate) pada 100 juta baris di dalam CTE.

Solusi Terbaik: Masukkan data ke *Temp Table* berindeks kluster (*Clustered Index*) terlebih dahulu sebelum menjalankan fungsi agregasi atau pemutaran data.

Gunakan Partitioned Tables Alih-alih UNION ALL

Penjelasan: Menggabungkan puluhan tabel atau subquery secara manual menggunakan UNION ALL membuat SQL Server kesulitan membuat *execution plan* yang efisien.

Contoh: SELECT * FROM Sales2024 UNION ALL SELECT * FROM Sales2025 UNION ALL ...

Solusi Terbaik: Manfaatkan fitur native *Table Partitioning* milik SQL Server.

Gunakan Table Hint NOLOCK / Read Uncommitted

Penjelasan: Saat membaca tabel operasional (OLTP) yang sedang aktif, kueri SSIS bisa tertahan oleh transaksi aplikasi lain (*blocking*).

Contoh: Kueri SSIS menggantung (*wait status*) menunggu proses penulisan data oleh aplikasi backend selesai.

Solusi Terbaik: Tambahkan hint WITH (NOLOCK) pada kueri sumber jika toleran terhadap *dirty read*, atau atur *Transaction Isolation Level* ke Read Uncommitted.

2. Konektivitas & Provider Data

Gunakan Provider Native OLE DB

Penjelasan: Mesin SSIS dirancang secara native menggunakan OLE DB. Penggunaan Provider ADO.NET atau ODBC menambah lapisan penerjemah data (*wrapper overhead*) yang memperlambat pemrosesan.

Contoh: Memilih *ADO.NET Connection Manager* untuk menghubungkan SSIS ke SQL Server.

Solusi Terbaik: Gunakan **MSOLEDBSQL** (*Microsoft OLE DB Driver for SQL Server*) pada seluruh *Connection Manager*.

Aktifkan DelayValidation = True

Penjelasan: Secara default, SSIS mengecek ketersediaan seluruh koneksi dan struktur tabel sebelum paket dijalankan. Jika koneksi jaringan lambat atau tabel bersifat sementara, validasi ini memakan waktu lama.

Contoh: Paket SSIS berhenti (*hang*) beberapa menit di awal eksekusi hanya untuk memvalidasi tabel *staging*.

Solusi Terbaik: Ubah properti DelayValidation = True pada level *Connection Manager* maupun *Data Flow Task*.

3. Desain & Komponen Data Flow (Canvas SSIS)

Eliminasi Komponen *Blocking*

Penjelasan: Komponen *blocking* (seperti Sort, Aggregate, atau Fuzzy Grouping) memuat seluruh baris data ke dalam RAM sebelum meneruskan baris pertama ke komponen berikutnya. Ini menghentikan alur kerja pipeline (*pipeline stall*).

Contoh: Menempatkan komponen Sort di tengah-tengah canvas SSIS Data Flow.

Solusi Terbaik: Hapus komponen Sort dari SSIS. Lakukan pengurutan data (ORDER BY) langsung di kueri SQL sumber.

Gunakan *Full Cache* pada Lookup

Penjelasan: Menggunakan mode *Partial Cache* atau *No Cache* pada komponen *Lookup* memaksa SSIS mengirim kueri individual ke database untuk setiap baris data yang lewat.

Contoh: Memproses 1 juta baris dengan *No Cache Lookup* menghasilkan 1 juta panggilan kueri SELECT ke server.

Solusi Terbaik: Atur *Lookup* ke mode **Full Cache**. Jika tabel acuan terlalu besar, tapis ukurannya menggunakan kueri SQL pada tab *Lookup Transformation*.

Manfaatkan *Shared Cache Connection Manager*

Penjelasan: Jika beberapa Data Flow Task memerlukan data acuan (*lookup*) dari tabel yang sama, konfigurasi standar akan mengunduh dan membangun cache tersebut berulang kali di RAM.

Contoh: Tiga *Data Flow Task* berbeda masing-masing mengunggah tabel dimensi berukuran 2 GB ke RAM.

Solusi Terbaik: Buat **Cache Connection Manager** terpisah. Isi data sekali saja menggunakan *Cache Transform*, lalu gunakan kembali cache tersebut di seluruh komponen *Lookup*.

Pastikan Data Benar-Benar Terurut pada Merge Join

Penjelasan: Komponen Merge Join membutuhkan input yang sudah terurut. Mengatur properti *IsSorted* = True tanpa adanya pengurutan nyata dari kueri akan menghasilkan data yang rusak atau kegagalan eksekusi.

Contoh: Mengubah properti *IsSorted* menjadi *True* pada *OLE DB Source Output* tanpa menuliskan klausa ORDER BY di SQL.

Solusi Terbaik: Sertakan klausa ORDER BY pada kueri SQL, lalu set properti *IsSorted* = True dan tentukan urutan kolom pada *SortKeyPosition*.

Pangkas Kolom Tidak Terpakai

Penjelasan: Kolom yang ikut mengalir dalam pipeline meski tidak digunakan akan memperlebar ukuran baris data (*buffer width*), sehingga kapasitas baris per buffer berkurang.

Contoh: Membawa kolom deskripsi panjang VARCHAR(8000) dari tabel asal padahal hanya memerlukan kolom ID dan Total.

Solusi Terbaik: Hapus centang pada kolom-kolom yang tidak diperlukan di tab *Columns* pada komponen *Source*.

Hindari Implicit Data Type Conversion

Penjelasan: Ketidakcocokan tipe data (seperti VARCHAR ke NVARCHAR atau DT_STR ke DT_WSTR) memicu konversi otomatis yang membebani CPU dan memicu kesalahan memori pada LOB/BLOB.

Contoh: Kolom sumber bertipe VARCHAR, namun dibaca sebagai DT_WSTR (Unicode) oleh SSIS.

Solusi Terbaik: Samakan tipe data langsung pada kueri SQL menggunakan CAST/CONVERT dan hindari penggunaan tipe data tak terbatas seperti VARCHAR(MAX) jika panjang data sebenarnya terbatas.

Hindari OLE DB Command Transformation

Penjelasan: Komponen ini mengeksekusi perintah SQL (seperti UPDATE atau INSERT) baris demi baris untuk setiap record yang melintas.

Contoh: Memproses 500.000 baris berarti mengeksekusi 500.000 kueri UPDATE terpisah secara berurutan.

Solusi Terbaik: Gunakan pendekatan dua langkah:

1. Muat data ke tabel *Staging* sementara menggunakan *OLE DB Destination (Fast Load)*.
2. Eksekusi satu kueri *set-based* UPDATE berbasis JOIN menggunakan **Execute SQL Task**.

Optimasi Script Component (.NET)

Penjelasan: Kode di dalam Script Component yang kurang terstruktur (seperti inisialisasi variabel atau koneksi di dalam fungsi pemroses baris) akan dieksekusi berulang jutaan kali.

Contoh: Membuka koneksi database atau membaca konfigurasi file di dalam method `Input0_ProcessInputRow`.

Solusi Terbaik: Pindahkan alokasi memori, pembukaan koneksi, atau pembacaan variabel ke method `PreExecute()`, lalu tutup koneksi di method `PostExecute()`.

Jalankan Data Flow Independen Secara Paralel

Penjelasan: Menjalankan beberapa Data Flow Task secara berurutan (*serial*) padahal pemrosesan datanya tidak saling berhubungan memperlama total waktu eksekusi.

Contoh: Menghubungkan *Data Flow Customer* ke *Data Flow Product* menggunakan garis hijau (*Precedence Constraint*).

Solusi Terbaik: Biarkan *Data Flow Task* yang independen berdiri tanpa garis penghubung (*unconnected tasks*) agar SSIS mengeksekusinya secara paralel.

4. Konfigurasi Buffer, Memori & Eksekusi Pipeline

Sesuaikan Ukuran Buffer

Penjelasan: Pengaturan standar SSIS (10,000 baris / 10 MB per buffer) sering kali terlalu kecil untuk spesifikasi server modern.

Contoh: Menggunakan batas buffer default 10 MB saat memproses puluhan juta baris data.

Solusi Terbaik: Aktifkan properti **AutoAdjustBufferSize** pada *Data Flow Task*, atau naikan `DefaultBufferSize` secara manual ke rentang 100 MB–200 MB.

Cegah Buffer Spooling ke Disk

Penjelasan: Jika alokasi RAM server tidak mencukupi, SSIS akan menulis buffer memori sementara ke harddisk (*spooling*), yang menurunkan performa pemrosesan secara drastis.

Contoh: Muncul pesan peringatan pada log: *"The buffer manager failed to allocate memory..."*

Solusi Terbaik: Sediakan RAM yang memadai, hilangkan komponen *blocking*, dan pastikan ukuran buffer muat di dalam memori fisik.

Arahkan Folder Temporary ke Disk Terpisah

Penjelasan: Saat terjadi *spooling* atau pemrosesan data besar (BLOB), SSIS secara bawaan menyimpan file sementara di drive OS (C:\).

Contoh: Kinerja server melambat akibat drive C:\ kehabisan ruang disk dan mengalami I/O bottleneck.

Solusi Terbaik: Atur properti `BLOBTempStoragePath` dan `BufferTempStoragePath` pada *Data Flow Task* ke drive penyimpanan lokal berkecepatan tinggi (SSD/NVMe khusus).

Optimasi Thread & Eksekusi Paralel

Penjelasan: Batasan thread yang tidak disesuaikan dengan jumlah core CPU membuat resource server tidak dimaksimalkan.

Contoh: Server memiliki 32 Core CPU, tetapi `MaxConcurrentExecutables` dibatasi hanya untuk 2 task.

Solusi Terbaik: Biarkan `MaxConcurrentExecutables` pada nilai -1 (otomatis mengikuti jumlah core CPU) dan sesuaikan properti `EngineThreads` pada *Data Flow Task*.

5. Tujuan / Loading ke Destination

Gunakan OLE DB Destination Fast Load

Penjelasan: Mode insersi standar memasukkan data baris demi baris, sedangkan mode *Fast Load* menggunakan perintah *bulk insert*.

Contoh: Memasukkan 1 juta baris memerlukan waktu beberapa jam karena dieksekusi sebagai 1 juta pernyataan INSERT terpisah.

Solusi Terbaik: Selalu pilih opsi **OLE DB Destination - Fast Load** pada *Data Access Mode*.

Sertakan Opsi TABLOCK

Penjelasan: Tanpa opsi TABLOCK, proses *bulk insert* akan mengunci data di tingkat baris atau halaman, yang menimbulkan beban overhead penguncian (*locking overhead*) serta mencegah fitur *minimal logging*.

Contoh: Opsi *FastLoadOptions* hanya berisi CHECK_CONSTRAINTS.

Solusi Terbaik: Tambahkan TABLOCK pada kolom *FastLoadOptions* (contoh: TABLOCK,CHECK_CONSTRAINTS).

Perbesar Commit Size

Penjelasan: Nilai *Maximum insert commit size* yang terlalu kecil memicu proses transaksi *commit* berulang kali ke log database.

Contoh: Mengatur commit size sebesar 100 pada total 10 juta baris data memicu 100.000 kali proses transaksi commit.

Solusi Terbaik: Biarkan pada nilai default 2147483647 (commit satu kali di akhir) atau atur ke angka besar seperti 100,000 hingga 500,000.

Nonaktifkan Indeks & Constraints Sementara

Penjelasan: Memasukkan data dalam jumlah besar ke tabel yang memiliki indeks aktif memaksa server merestrukturisasi indeks (*B-Tree rebalance*) pada setiap baris yang masuk.

Contoh: Memuat 20 juta baris data ke tabel yang memiliki 5 *Non-Clustered Index*.

Solusi Terbaik:

1. *Disable* atau *Drop Non-Clustered Index* sebelum proses muat data.
2. Jalankan proses pemuatan data (*bulk load*).
3. Lakukan *Rebuild* indeks setelah pemuatan selesai.

Matikan Trigger Destination

Penjelasan: Keberadaan AFTER INSERT trigger pada tabel tujuan akan memicu eksekusi logika trigger pada setiap batch data, yang mematikan efisiensi fitur *Fast Load*.

Contoh: Trigger pencatatan log audit berjalan pada setiap baris baru yang di-insert.

Solusi Terbaik: Nonaktifkan trigger sementara sebelum ETL berjalan (DISABLE TRIGGER ALL ON TargetTable), lalu aktifkan kembali setelah proses selesai.

6. Arsitektur, Orkestrasi Job & Lingkungan Server

Gunakan Pola Arsitektur Parent-Child

Penjelasan: Sebuah paket SSIS raksasa (*monolithic*) yang berisi puluhan Data Flow memakan memori metadata yang besar dan sulit dikelola saat terjadi kesalahan.

Contoh: Satu paket SSIS berukuran 50 MB yang menjalankan seluruh alur ETL Data Warehouse.

Solusi Terbaik: Bagi paket menjadi modul-modul kecil terpisah (*child packages*), lalu panggil menggunakan **Execute Package Task** dari sebuah paket utama (*parent package*).

Terapkan Checkpoints

Penjelasan: Tanpa *checkpoint*, kegagalan eksekusi di akhir proses memaksa seluruh alur ETL diulang kembali dari awal.

Contoh: Paket ETL yang berjalan selama 2 jam gagal pada menit ke-110, sehingga proses jam pertama harus diulang kembali.

Solusi Terbaik: Aktifkan fitur *Checkpoint* pada Control Flow (`SaveCheckpoints = True`, `CheckpointUsage = IFEXISTS`) agar SSIS dapat melanjutkan (*resume*) eksekusi dari task terakhir yang gagal.

Atur Urutan Eksekusi Job (Job Chaining)

Penjelasan: Beberapa job SSIS yang berjalan bersamaan dapat saling memperebutkan resource CPU, disk I/O, atau memicu penguncian tabel (*deadlock*).

Contoh: Job ETL Penjualan dan Job ETL Stok barang dijadwalkan berjalan bersamaan pada pukul 01.00 dan mengakses tabel staging yang sama.

Solusi Terbaik: Atur urutan eksekusi berantai (*Job Chaining*) pada SQL Server Agent atau beri jeda jadwal yang aman.

Gunakan Incremental Load

Penjelasan: Menghapus dan memuat ulang seluruh isi tabel (*Full Reload*) setiap hari tidak efisien jika perubahan data harian hanya sebagian kecil.

Contoh: Memuat ulang seluruh data histori transaksi selama 5 tahun terakhir setiap malam.

Solusi Terbaik: Terapkan pola **Incremental Load** menggunakan kolom penanda waktu (`LastModifiedDate`), *Change Data Capture* (CDC), atau *Change Tracking*.

Optimasi tempdb Database

Penjelasan: tempdb adalah pusat pemrosesan operasi pengurutan dan pencatatan tabel sementara. Jika konfigurasi tempdb tidak optimal, seluruh proses ETL akan terhambat.

Contoh: Database tempdb hanya memiliki 1 file data dan berada di drive C:\ bersama sistem operasi.

Solusi Terbaik: Buat jumlah file data tempdb sesuai jumlah core CPU (maksimal 8 file awal dengan ukuran seimbang) dan tempatkan di media penyimpanan berkecepatan tinggi.

Alokasikan RAM/CPU Memadai

Penjelasan: Spesifikasi hardware yang terbatas tidak mampu menampung pemrosesan paralel skala besar, sehingga memicu persaingan memori antara SQL Server dan SSIS.

Contoh: Memproses ETL skala besar pada server dengan RAM 8 GB.

Solusi Terbaik: Tingkatkan kapasitas hardware dan atur batas *Max Server Memory* pada SQL Server agar menyisakan ruang RAM minimal 4 GB–8 GB khusus untuk sistem operasi dan mesin SSIS (DTE_{exec}.exe).

Pengecualian Antivirus (Exclusions)

Penjelasan: Fitur pemindaian antimalware secara real-time yang memindai file data dan folder sementara SSIS saat aktivitas *read/write* intensif dapat menurunkan kecepatan pemrosesan.

Contoh: Windows Defender memindai buffer sementara SSIS setiap kali terjadi penulisan data.

Solusi Terbaik: Tambahkan pengecualian proses (DTE_{exec}.exe, sqlservr.exe) serta pengecualian folder (folder data SQL Server & folder Temp SSIS) pada konfigurasi antivirus server.

Gunakan Logging Level Ringan di Production

Penjelasan: Mengatur tingkat pencatatan log SSIS Catalog (SSISDB) ke mode Verbose atau Diagnostic di lingkungan produksi menghasilkan I/O penulisan log yang sangat tinggi.

Contoh: SSIS mencatat informasi detail per buffer ke database SSISDB, yang menyebabkan database log membengkak dan eksekusi memanjang.

Solusi Terbaik: Gunakan tingkat logging **Basic** atau **Performance** untuk operasional harian di lingkungan Production, dan gunakan mode Verbose hanya saat melakukan pelacakan masalah (*troubleshooting*).